Queen Mary
University of London
Science and Engineering

EBU4202: Digital Circuit Design
Number Systems & Codes Tutorials

Dr. Md Hasanuzzaman Sagor (Hasan)
Dr. Chao Shu (Chao)
Dr. Farha Lakhani (Farha)
Dr. Fatma Benkhelifa

School of Electronic Engineering and Computer Science,
Queen Mary University of London,
London, United Kingdom.

Overview: Number Systems & Codes

* Introduction
* Number Systems
* Binary and Hex Arithmetic
* Negative Numbers & Floating Point
* Codes: Binary, Decimal, Alphanumeric

Chapters 1 & 2 – "Digital Design: Principles and Practices" book

Positional Number Systems

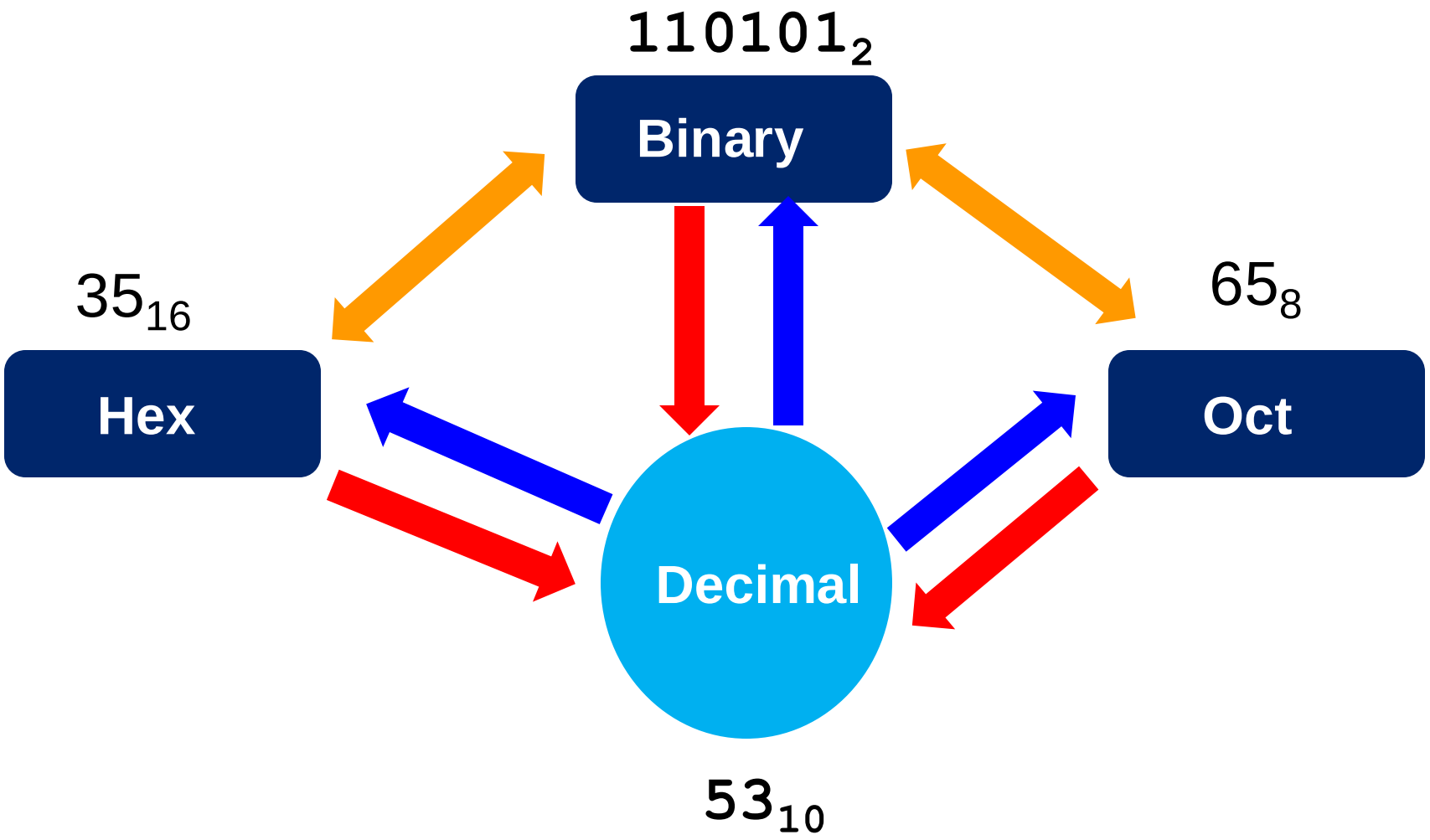
http://m.news.cn/bj/2022-01/05/c_1128232693_3.htm

Time is written using a SEXAGESIMAL system (base 60),
But the digits are written using the DECIMAL system.

- Each number is represented by a string of digits; each digit position has an associated weight.
- The value of the number is equal to the weighted sum of all digits, with the weights determined by the digit's position and the base (or radix) of the numbering system.

Queen Mary
University of London

Conversion



Digits in base **R** range from **0** to **(R – 1)**.

Conversion: Any Base to Decimal

- **Warm-Up Example**: Conversion of binary to decimal

  $10011_2 = ?$

- **Example**: Conversion of base 5 to decimal

  $4321_5 = ?$

- **Example**: Conversion of base 5 to decimal

  $12.3_5 = ?$

base 5 will have digits in the range 0 to (5-1)=4.

Trick: Conversion to Decimal

- ***Example***: 110110111001_2 to decima

Solution:

$$
\begin{aligned}
110110111001_2 &= 1101 \quad 1011 \quad 1001 \\
&= D \quad B \quad 9 \\
&= (13 \times 16^2) + (11 \times 16^1) + (9 \times 16^0) \\
&= (13 \times 256) + (11 \times 16) + (9 \times 1) \\
&= 3328 + 176 + 9 \\
&= 3513_{10}
\end{aligned}
$$

- **Example**: Conversion of binary to octal

$$101110_2 = (?)_8$$

Review: Subtraction with 2's Complement

This can be rewritten as: $23_{10} + (-45_{10})$, so we need to convert the binary representation of 45_{10} to two's complement.

45_{10} = 0010 1101

1101 0010 ← Invert

+0000 0001 ← Add 1

1101 0011 = -45_{10}

Now just add!

	23	0001 0111
+	-45	+1101 0011
	-22	1110 1010
		-0000 0001 ← Subtract 1
		1110 1001
		0001 0110 ← Invert to convert to simple binary

Check

This is -22_{10} really, as we're just checking!

$= 1*2^4 + 0*2^3 + 1*2^2 + 1*2^1 + 0*2^0$

$= 16 + 0 + 4 + 2 + 0 = 22_{10}$

To subtract two signed numbers, take the 2's complement of the subtrahend and add. Discard any final carry bit.

Example: Subtraction with 2's Complement

Example 1: Perform the following subtraction of signed numbers: $15_{10} - 6_{10}$

```
6₁₀ = 0000 0110
      1111 1001   ← Invert
      0000 0001   ← Add 1
      ----------
      1111 1010 = -6₁₀
```

```
     15    0000 1111
+   - 6   +1111 1010
-----------------------
      9  1  0000 1001
```

Discard

Example: Subtraction with 2's Complement

Example 2: Perform the following subtraction of signed numbers:
00001100 - 11110111 [$12_{10} - (-9_{10})$]

Solution:

In this case, 12 – (– 9) = 12 + 9 = 21.

$-(-9)_{10}$ = 1111 0111
0000 1000 ← Invert
0000 0001 ← Add 1
0000 1001 = 9_{10}

	12	0000 1100
+	9	+0000 1001
	21	0001 0101

Example: Subtraction with 2's Complement

Example 3: Perform the following subtraction of signed numbers:
11100111 – 00010011 [– 25_{10} – (+ 19_{10})]

Solution:

In this case, – 25_{10} – (+ 19_{10}) = – 25 + (– 19) = – 44.

19_{10} = 0001 0011
1110 1100 ← Invert
0000 0001 ← Add 1
1110 1101 = $(-19)_{10}$

	–25	1110 0111
+	–19	+1110 1101
	–44	1 1101 0100

Discard

Example (Review): IEEE-754 FP

- Represent 0.0390625_{10} in floating point format:

- Represent -0.75_{10} in floating point format:

- Represent 145.84375_{10} in floating point format:

Example (3/3): IEEE-754 FP

Convert the IEEE-754 floating point number

x = 01000011010111010000000000000000 [16 zeros] to a decimal number:

Solution:

Exponent: $10000110_2 = 134_{10}$

Mantissa: *10111010000000000000000*

$$x = 1.1011101_2 \times 2^{(134-127)}$$

$$= 11011101_2$$

$$= 221_{10}$$

Example: Odd and Even Parity

What is actually sent when even parity has been agreed.

	'S'	'E'
ASCII	101 0011	100 0101
Even parity	0101 0011	1100 0101
Odd Parity	1101 0011	0100 0101

- **Example** (detection of a 1-bit error):
  - ASCII '**S**' is sent (i.e., send $\mathbf{1010011_2}$), but value $\mathbf{01010010_2}$ is received.

Assuming value 01010010_2 is received, can Parity Checking detect an error when character 'S' is sent, if *even parity* has been agreed?

sent = 01010011 ('S')
received = 01010010
Received value has odd number of 1's (so doesn't obey even parity), so error is detected (1 bit flipped).

Example: Odd and Even Parity

What is actually sent when even parity has been agreed.

	'S'	'E'
ASCII	101 0011	100 0101
Even parity	0101 0011	1100 0101
Odd Parity	1101 0011	0100 0101

- **Example** (detection of a 1-bit error):
  - ASCII '**E**' is sent (i.e., send $\mathbf{1000101_2}$), but value $\mathbf{01010010_2}$ is received.

What if instead we send character 'E' and *odd parity* has been agreed? Can Parity Checking detect an error?

sent = 01000101 ('E')
received = 01010010 → ASCII character 'R'
Received value has an odd number of 1's (so obeys odd parity), so no error is detected (there are 4 bits flipped).

Example: Odd and Even Parity

What is actually sent when even parity has been agreed.

	'S'	'E'
ASCII	101 0011	100 0101
Even parity	0101 0011	1100 0101
Odd Parity	1101 0011	0100 0101

- **Exercise** (detection of a 1-bit error):
  - ASCII '**E**' is sent (i.e., send $\mathbf{1000101_2}$) and even parity has been agreed, but value $\mathbf{00000101_2}$ is received.

sent = 11000101 ('E')
received = 00000101 → ASCII character 'R'
Received value has an even number of 1's (so obeys even parity), so no error is detected (there are 2 bits flipped).